Chapter 03

Dependency management, Exception Handling, Logging

The examples are publicly available on:
https://github.com/skanderturki/se411

Outline

- Dependency management
- Exception handling
- Logging

Learning Outcomes

- CLO1: Define and understand the fundamentals of software construction concepts
- CLO4: Evaluate the key construction concepts in terms of best practices
- CLO5: Analyze and improve existing or new software projects using various software research techniques to ensure quality and innovation

Learning Outcomes

At the end of this topic, you should be able to understand the following concepts and their roles in software construction and

- Dependency management
- Exception handling
- Logging

and should be able to apply them appropriately in improving existing or developing new software projects

Dependency Management

Introduction to Dependency Management

- Dependency management refers to the process of identifying, tracking, and controlling the dependencies between different components of a software system.
- Managing dependencies is a difficult task that can be time consuming if done manually. The challenges include
  - How to make sure all the team members are using the same version of some library?
  - How to allow a developer to setup a new development environment without having to search/download/install/configure all the dependencies of the project?
  - How to easily manage the migration to new versions of dependencies at the level of the project configuration of all team members without time loss?

Introduction to Dependency Management (2)

- Dependency management tools allow to automate these tasks by providing a **declarative way** of specifying the project dependencies.

- Maven and Gradle are the most popular tools to manage dependencies in Java projects.





Maven

- Maven is a project management and build automation tool used by most Java developers.
- It utilizes a project object model (POM) to centralize and efficiently manage a project’s build process, reporting and documentation.
- When you use Maven, you describe your project using a well-defined project object model.
- Maven can then apply crosscutting logic from a set of shared (or custom) plugins.

Maven

- Maven encompasses
  - a POM (pom.xml),
  - set of standards,
  - a project lifecycle,
  - a dependency management system and
  - logic for executing plugin goals at defined phases in a lifecycle.
- Unlike some other tools, Maven offers more advanced features, making it an important tool for streamlining project management, dependency handling and documentation.

Defining a dependency in Maven

The pom.xml contains all the dependencies to (libraries and plugins).

Plugins are scripts that are executed by Maven

Plugins define **goals**: the **javafx** plugin defines the **run** goal (**javafx:run**).

```
<project …>
    <properties>
       <javafx.version>23</javafx.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.openjfx</groupId>
            <artifactId>javafx-controls</artifactId>
            <version>${javafx.version}</version>
        </dependency>
    </dependencies>
</project>
```

Here, we are configuring our project to use version 23 of JavaFX as a dependency, the library will be automatically downloaded by Maven.

Maven build lifecycle

- It constitutes a series of stages, also called phases, that get executed in the same order.
- Every Maven project has the following three built-in lifecycles
  - ***default***: This **lifecycle** handles the compiling, packaging, and deployment of a Maven project.
  - ***clean***: This **lifecycle** handles the deletion of temporary files and generated artifacts from the target directory.
  - ***site***: This **lifecycle** handles the generation of documentation and site generation.

Maven build lifecycle

These are Maven **phases** of the **default** lifecycle (the most commonly used lifecycle).

- ***validate***: Runs checks to ensure that the project is correct and that all dependencies are downloaded and available.
- ***compile***: Compiles the source code.
- ***test***: Runs unit tests using frameworks. This step does not require the application to be packaged.
- ***package***: Assembles compiled code into a distributable format, such as JAR or WAR.
- ***verify***: Performs additional checks on the packaged code to ensure it is valid.
- ***install***: Installs the packaged archive into a local repository. The archive is now available for use by any project running on that machine.
- ***deploy***: Pushes the built archive into a remote repository for use by other teams and team members.

Maven build lifecycle

These are Maven **phases** of the **clean** lifecycle.

- ***pre-clean:*** Executes processes before cleaning
- ***clean:*** Removes files generated by the previous build
- ***post-clean:*** Executes processes after cleaning

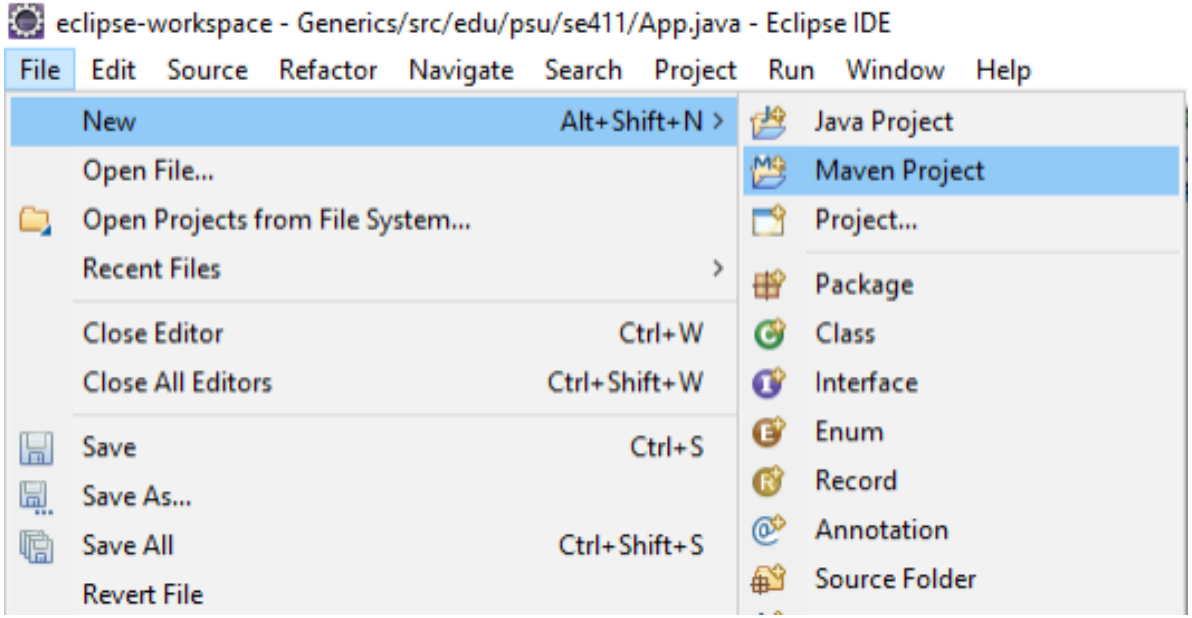
Maven build lifecycle

These are Maven **phases** of the **site** lifecycle.

- ***pre-site:*** Executes processes before generating site documentation
- ***site:*** Generates the site documentation
- ***post-site:*** Executes processes after generating site documentation
- ***site-deploy:*** Deploys the generated site documentation to a web server

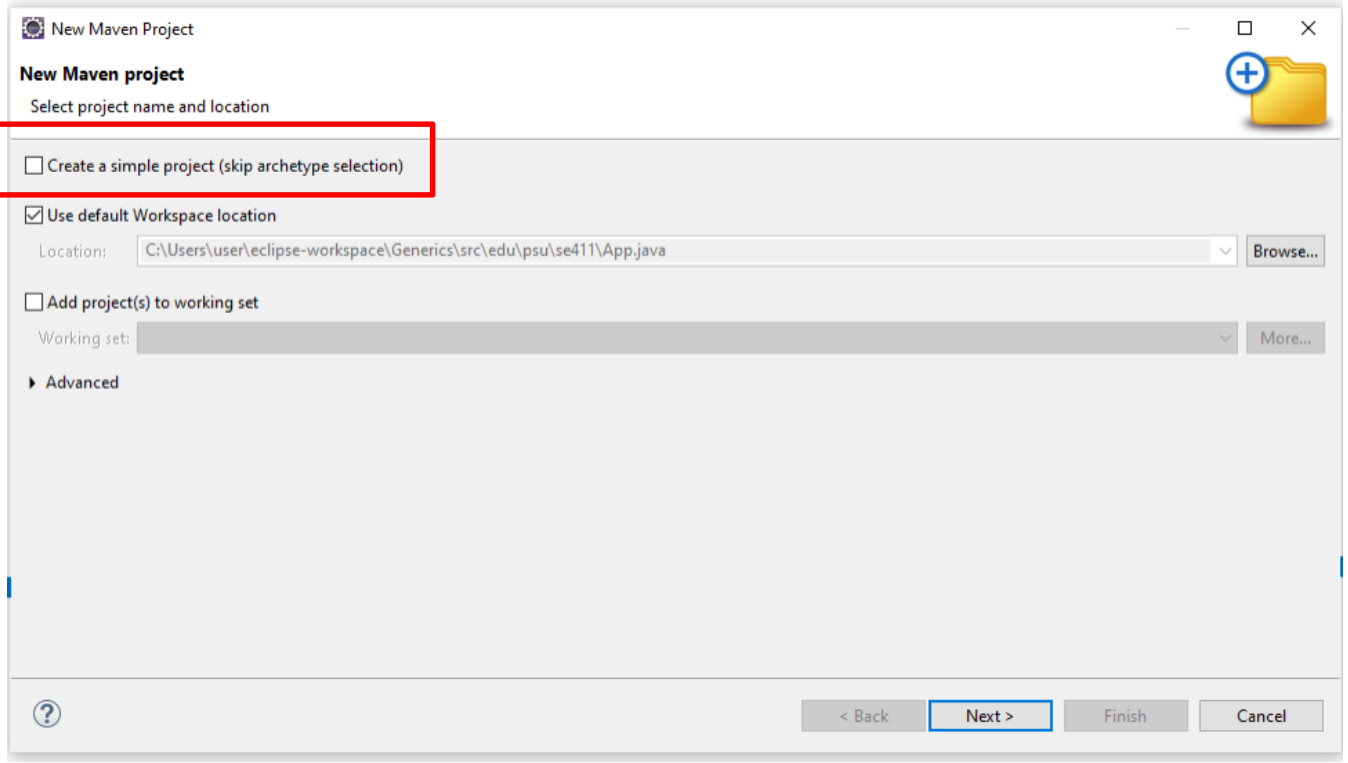
Maven with Eclipse

- Create a new Maven project via File → New → Maven Project



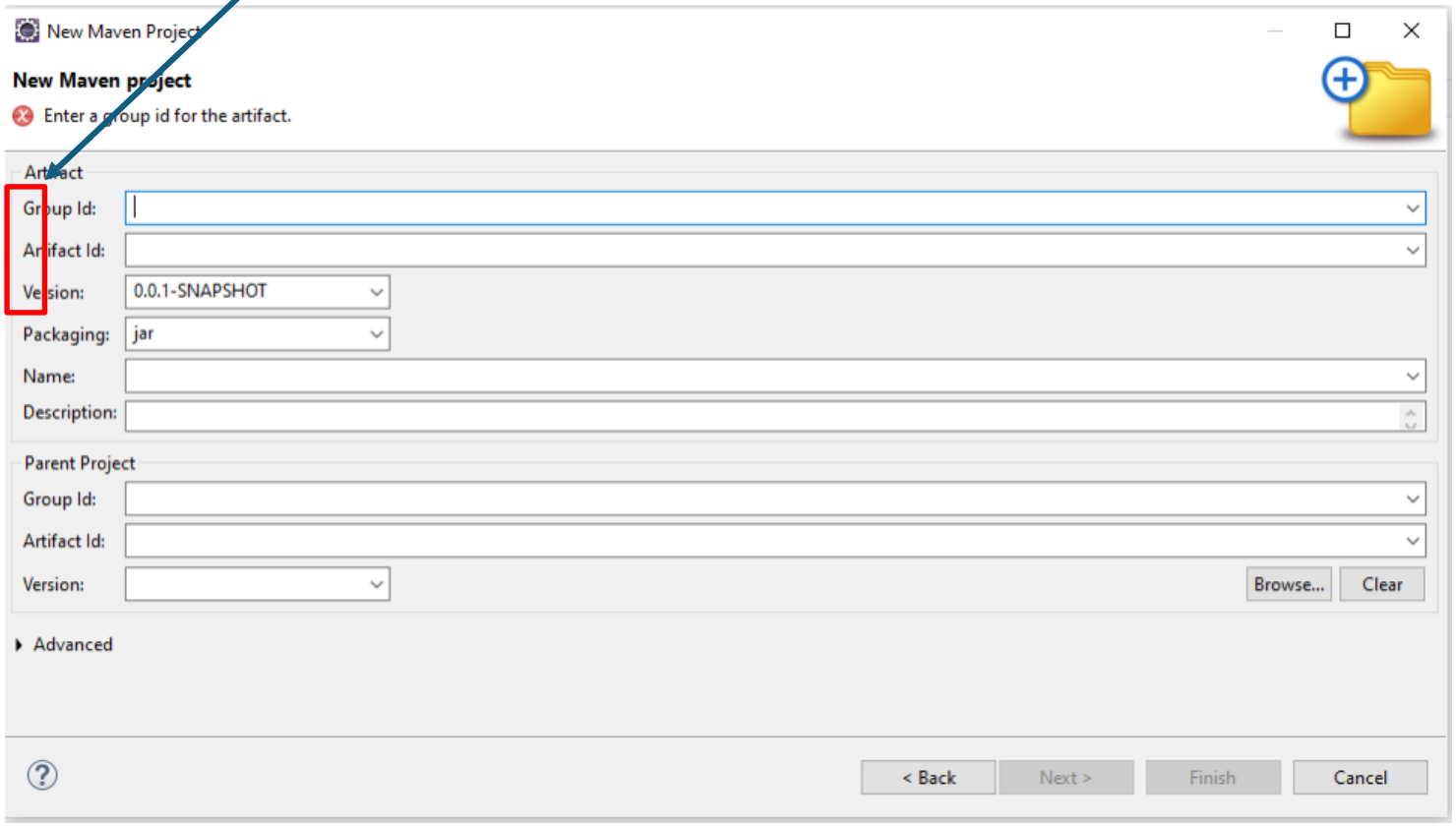
Maven with Eclipse

- On the first wizard page, you can select if you want to create a simple project or if you want to use an archetype as template for our project creation.



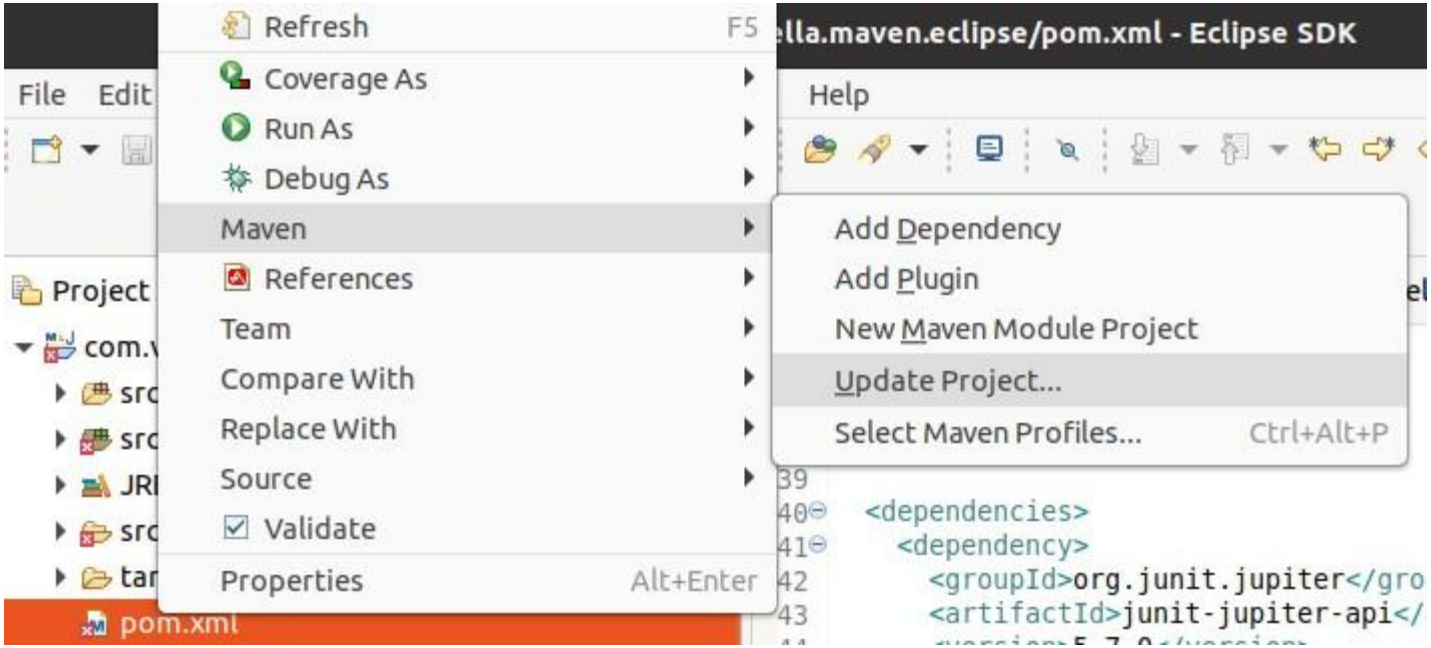
Maven with Eclipse

- Enter the GAV of your project similar to the following screenshot.



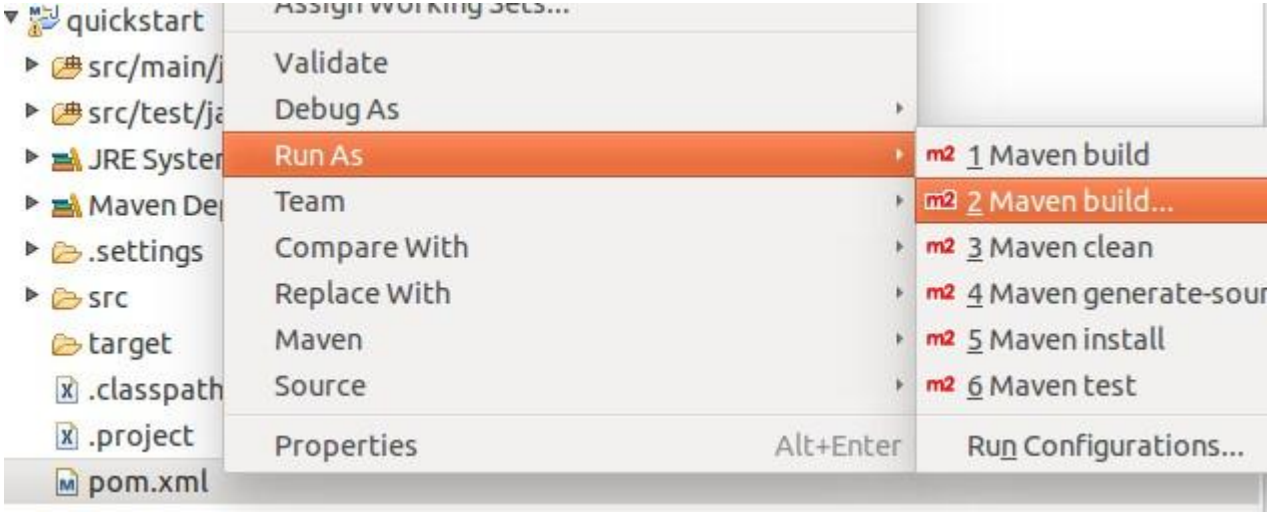
Maven with Eclipse

- Update your project when changes are made. Right click the POM file → Maven → Update Project…

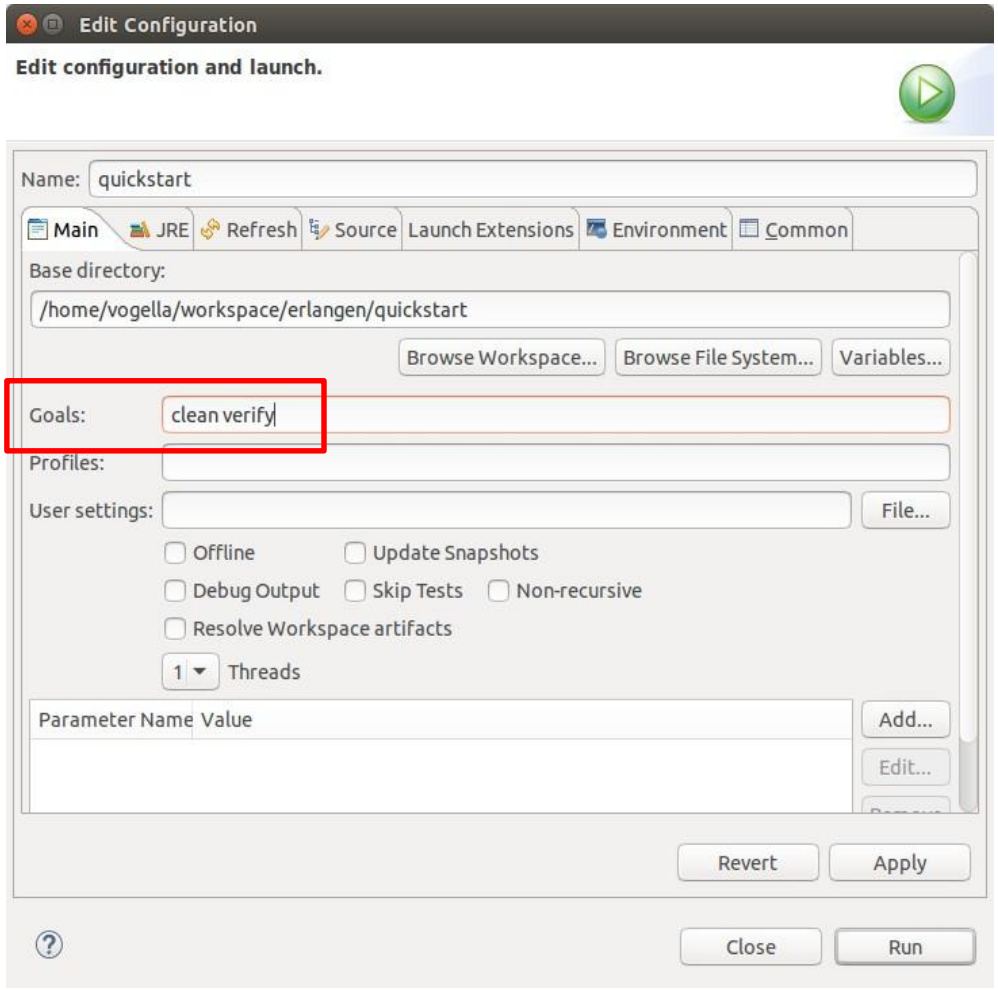


Maven with Eclipse

- Validate that the Maven build is correctly configured by running the build. Right click the POM file → Run As → Maven build…

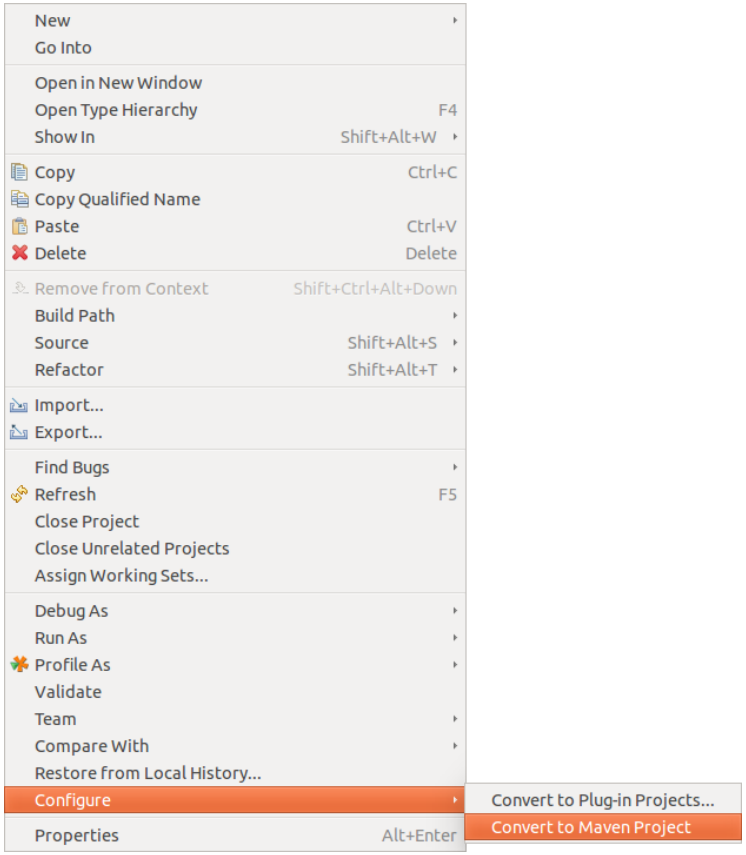


Maven with Eclipse



Maven with Eclipse

- You can also convert ordinary Java project to Maven project.



Example of core plugins: site plugin

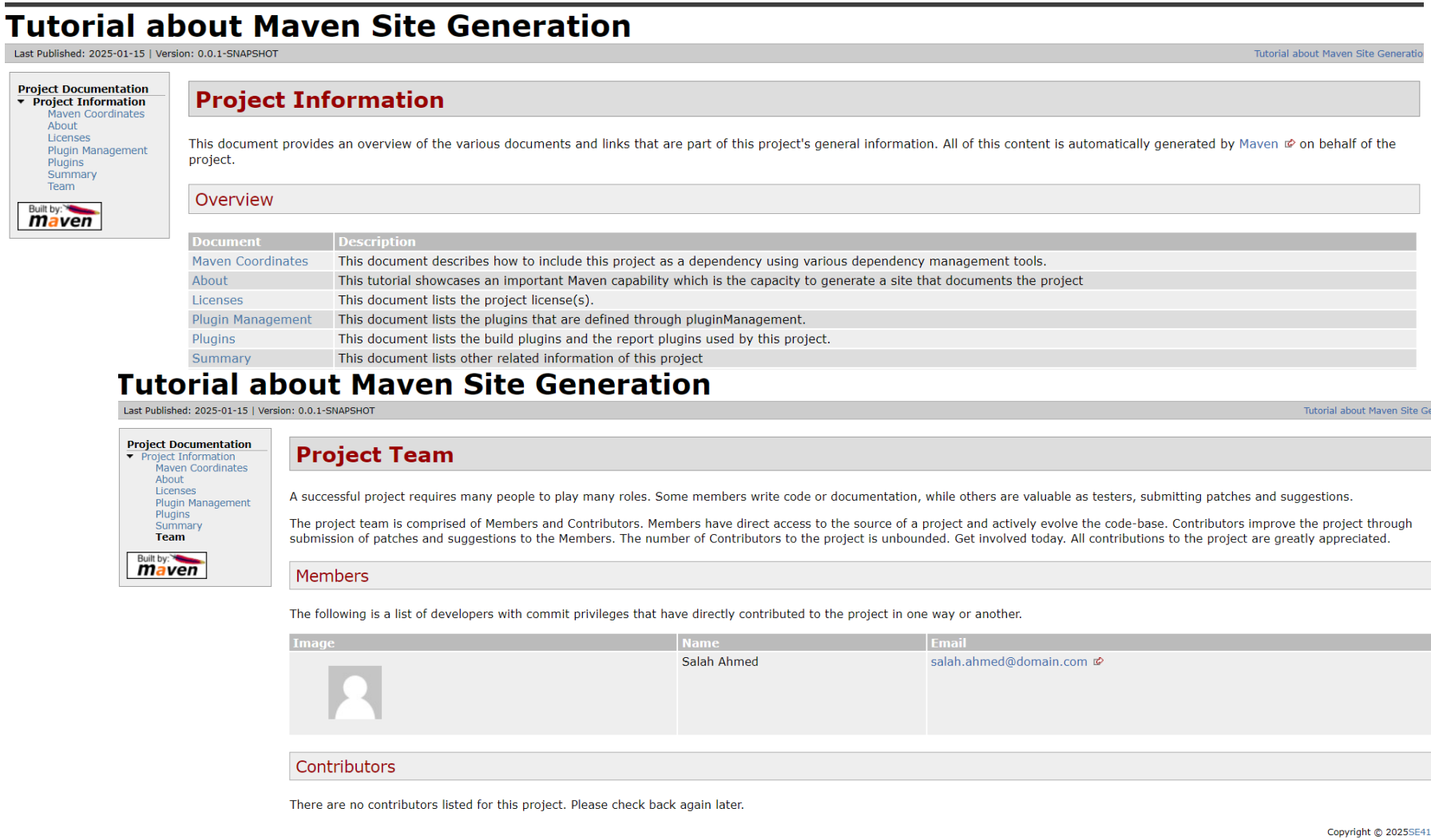
```
<name>Tutorial about Maven Site Generation</name>
<description>This tutorial showcases an important Maven capability which is the capacity to generate a site that documents the project</description>
<url>https://sites.google.com/view/se411/home</url>
<licenses>
    <license>
      <name>MIT License</name>
      <comments>This is an open-source project</comments>
    </license>
</licenses>
<organization>
    <name>SE411-Tech</name>
    <url>https://github.com/skanderturki/se411</url>
</organization>
<developers>
    <developer>
        <name>Salah Ahmed</name>
        <email>salah.ahmed@domain.com</email>
    </developer>
</developers>
```

In pom.xml, we can specify details about the project

Source Code: maven-02-site

Example of plugins: core site plugin

- After running the site goal:



Tutorial about Maven Site Generation

Last Published: 2025-01-15 | Version: 0.0.1-SNAPSHOT — Tutorial about Maven Site Generatio

Project Documentation
- **Project Information**
  - Maven Coordinates
  - About
  - Licenses
  - Plugin Management
  - Plugins
  - Summary
  - Team

Built by: maven

Project Information

This document provides an overview of the various documents and links that are part of this project's general information. All of this content is automatically generated by Maven on behalf of the project.

Overview

Document	Description
Maven Coordinates	This document describes how to include this project as a dependency using various dependency management tools.
About	This tutorial showcases an important Maven capability which is the capacity to generate a site that documents the project
Licenses	This document lists the project license(s).
Plugin Management	This document lists the plugins that are defined through pluginManagement.
Plugins	This document lists the build plugins and the report plugins used by this project.
Summary	This document lists other related information of this project

Tutorial about Maven Site Generation

Last Published: 2025-01-15 | Version: 0.0.1-SNAPSHOT — Tutorial about Maven Site Generation

Project Documentation
- Project Information
  - Maven Coordinates
  - About
  - Licenses
  - Plugin Management
  - Plugins
  - Summary
  - **Team**

Built by: maven

Project Team

A successful project requires many people to play many roles. Some members write code or documentation, while others are valuable as testers, submitting patches and suggestions.

The project team is comprised of Members and Contributors. Members have direct access to the source of a project and actively evolve the code-base. Contributors improve the project through submission of patches and suggestions to the Members. The number of Contributors to the project is unbounded. Get involved today. All contributions to the project are greatly appreciated.

Members

The following is a list of developers with commit privileges that have directly contributed to the project in one way or another.

Image	Name	Email
	Salah Ahmed	salah.ahmed@domain.com

Contributors

There are no contributors listed for this project. Please check back again later.

Copyright © 2025SE411-Tech. .

Example plugin: Create an executable

```
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-shade-plugin</artifactId>
            <version>3.5.1</version>
            <executions>
                <execution>
                        <phase>package</phase>
                        <goals>
                          <goal>shade</goal>
                        </goals>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
```

This plugin (using the shade goal) will create a “fat” executable (at the **package** phase) that contains all the required libraries embedded

```
\2024 Spring\SE411> java -classpath .\maven-03-dependencies-0.0.1-SNAPSHOT.jar psu.edu.se411.App
```

Source Code: maven-03-dependencies

Maven - Advantages

- **Dependency Management:** Simplifies the process of managing project dependencies. By defining dependencies in a single configuration file, Maven automatically downloads and includes the required libraries, ensuring consistency across projects.
- **Multi module Projects:** Supports multi module projects, enabling developers to manage large applications with various modules effectively. This organization allows for better separation of concerns and easier dependency management across modules.
- **Plugin Ecosystem:** Has a rich ecosystem of plugins that extend its functionality. These plugins can perform various tasks, such as code analysis, documentation generation, and deployment, allowing for highly customizable build processes.

Maven - Advantages

- **Easy to Learn:** Straightforward and declarative approach to project configuration using XML. This simplicity allows newcomers to learn it quickly and focus on development rather than build complexities.
- **Community Support:** Widely used in the industry, which means there is ample community support. Developers can easily find resources, documentation, and forums to resolve issues or learn best practices.
- **Integration with IDEs:** Popular integrated development environments (IDEs) such as Eclipse and IntelliJ IDEA have built in support for Maven, which enhances developer productivity through features like automatic dependency resolution and build management.

Exception Handling

Exception handling

- What is Exception? In Java, it is
  - an *event* that disrupts the normal flow of the program
  - an object which is thrown at runtime
- Exception Handling is a mechanism to handle runtime errors such as ClassNotFound, IO, SQL.
- The exception handling in java is one of the powerful mechanism to handle the runtime errors so that normal flow of the application can be maintained.

Exception handling: Motivations

- When a program runs into a runtime error, the program terminates abnormally.
- How can you handle the runtime error so that the program can **continue to run or terminate gracefully**?
- The answer is → Exception handling
- The main purpose of exception handling is to maintain the normal flow of an application/program as Exception normally disrupts its normal flow

Common exception scenarios

- ArithmeticException
  - E.g. divide by zero
- NullPointerException
  - E.g. performing any operation by the variable with null value
- NumberFormatException
  - E.g. wrong formatting of any value, such as converting string to digit
- ArrayIndexOutOfBoundsException
  - If you are inserting any value in the wrong index

Exception-Handling Overview

Example: Here we get a runtime error

```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    System.out.print("Enter two integers: ");

    int number1 = input.nextInt();
    int number2 = input.nextInt();

    // if number2 is 0, we get a runtime error
    System.out.println(number1 + " / " + number2 + " is " + (number1 / number2));
}
```

Fix it using an if statement

```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in); // Prompt the user to enter two integers
    System.out.print("Enter two integers: ");
    int number1 = input.nextInt();
    int number2 = input.nextInt();
    if (number2 != 0)
        System.out.println(number1 + " / " + number2 + " is " + (number1 / number2));
    else
        System.out.println("Divisor cannot be zero"); }
}
```

Exception Advantages

In the previous example we can write if..then..else because we know what error can occur.

In other, more complex scenarios, we cannot know which error may happen, for example when we connect to a database, write into a file, read from some hardware, etc.

In such situations, **exceptions** are the solution.

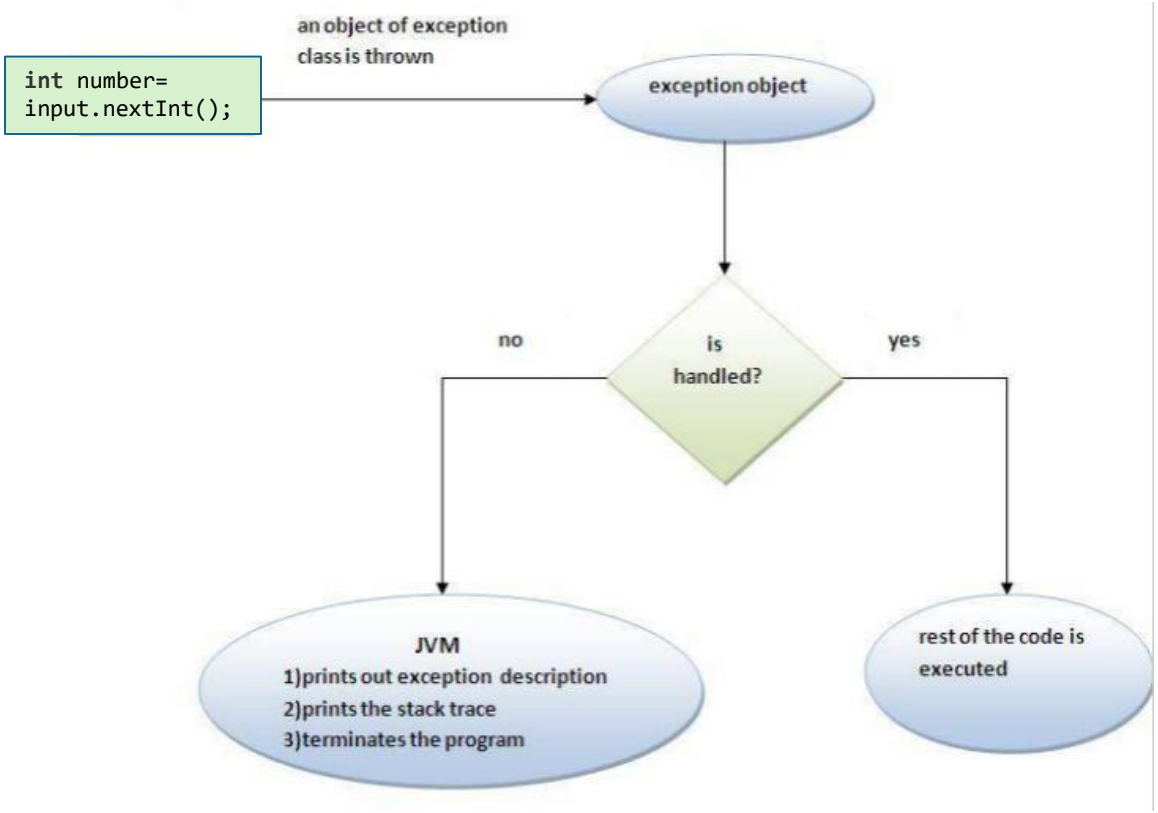
Example: Handling InputMismatchException

By handling **InputMismatchException**, your program will continuously read an input until it is correct.

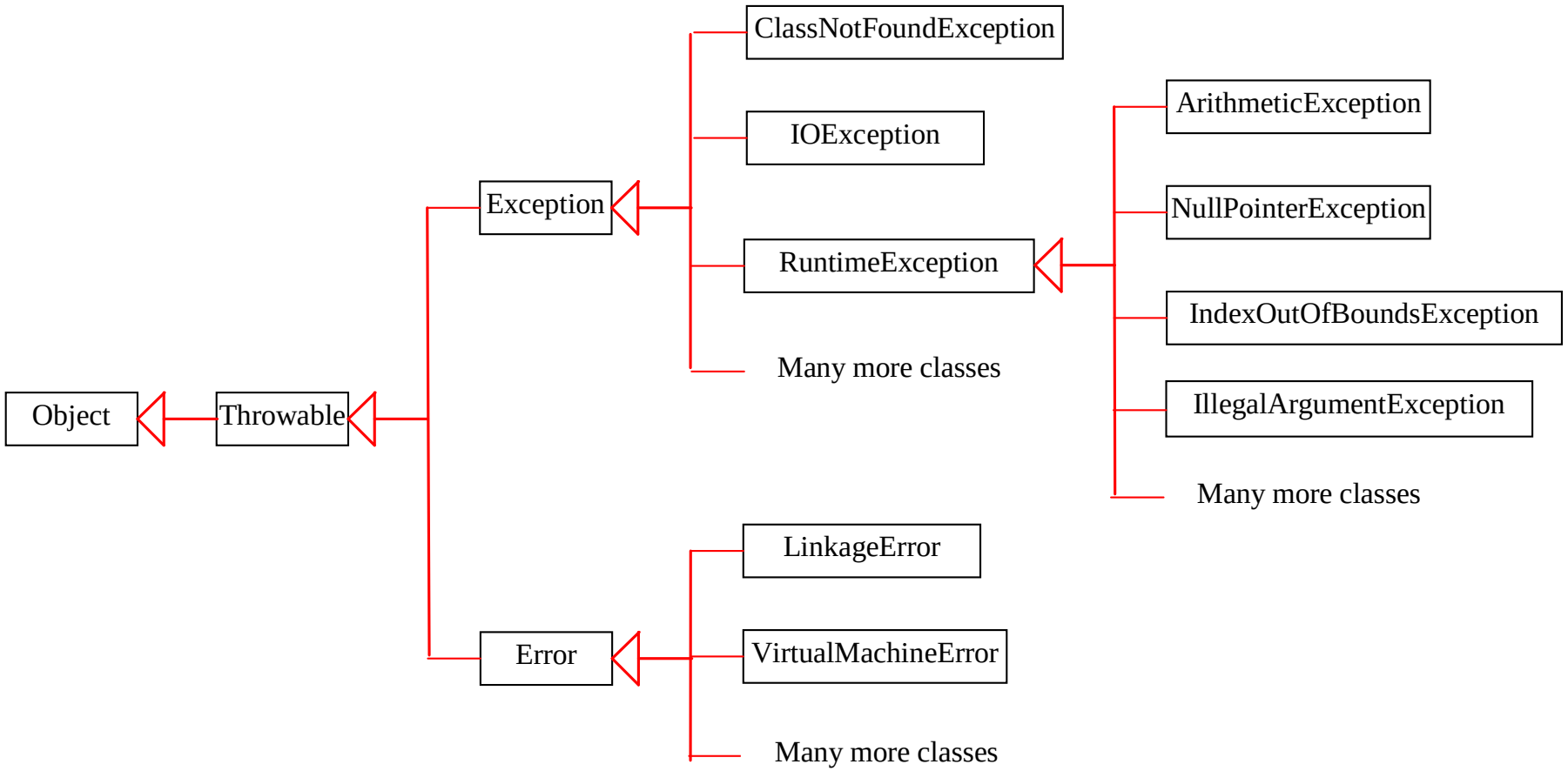
```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    boolean continueInput = true;
    do {
        try {
            System.out.print("Enter an integer: ");
            int number = input.nextInt();
            // Display the result
            System.out.println( "The number entered is " + number);
            continueInput = false;
        } catch (InputMismatchException ex) {
            System.out.println("Try again. (" + "Incorrect input: an integer is required)");
            input.nextLine(); // discard input
        }
    } while (continueInput);
}
```

Note: the developer needs to know which exception class to use.

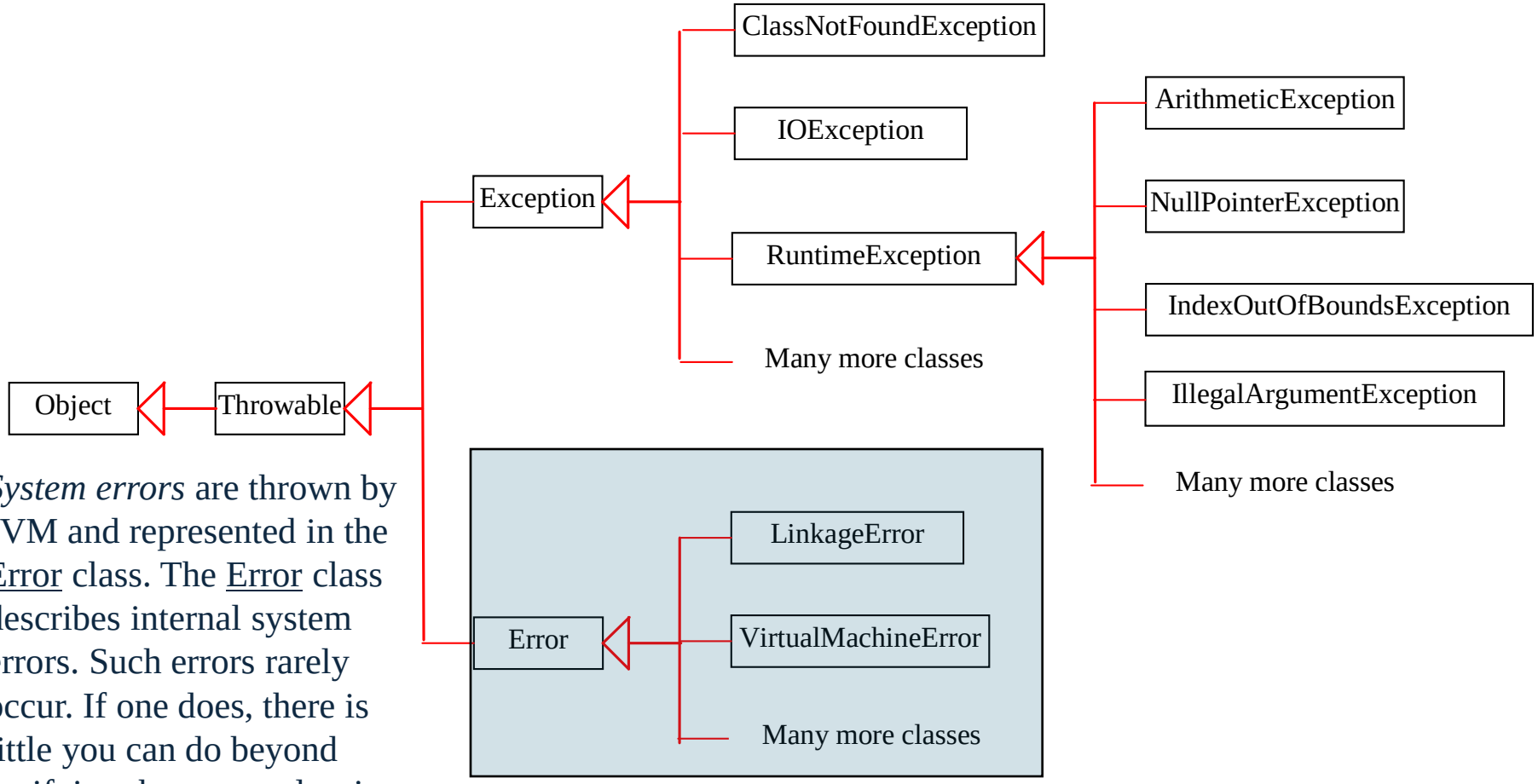
Example: Handling InputMismatchException



Exception Types



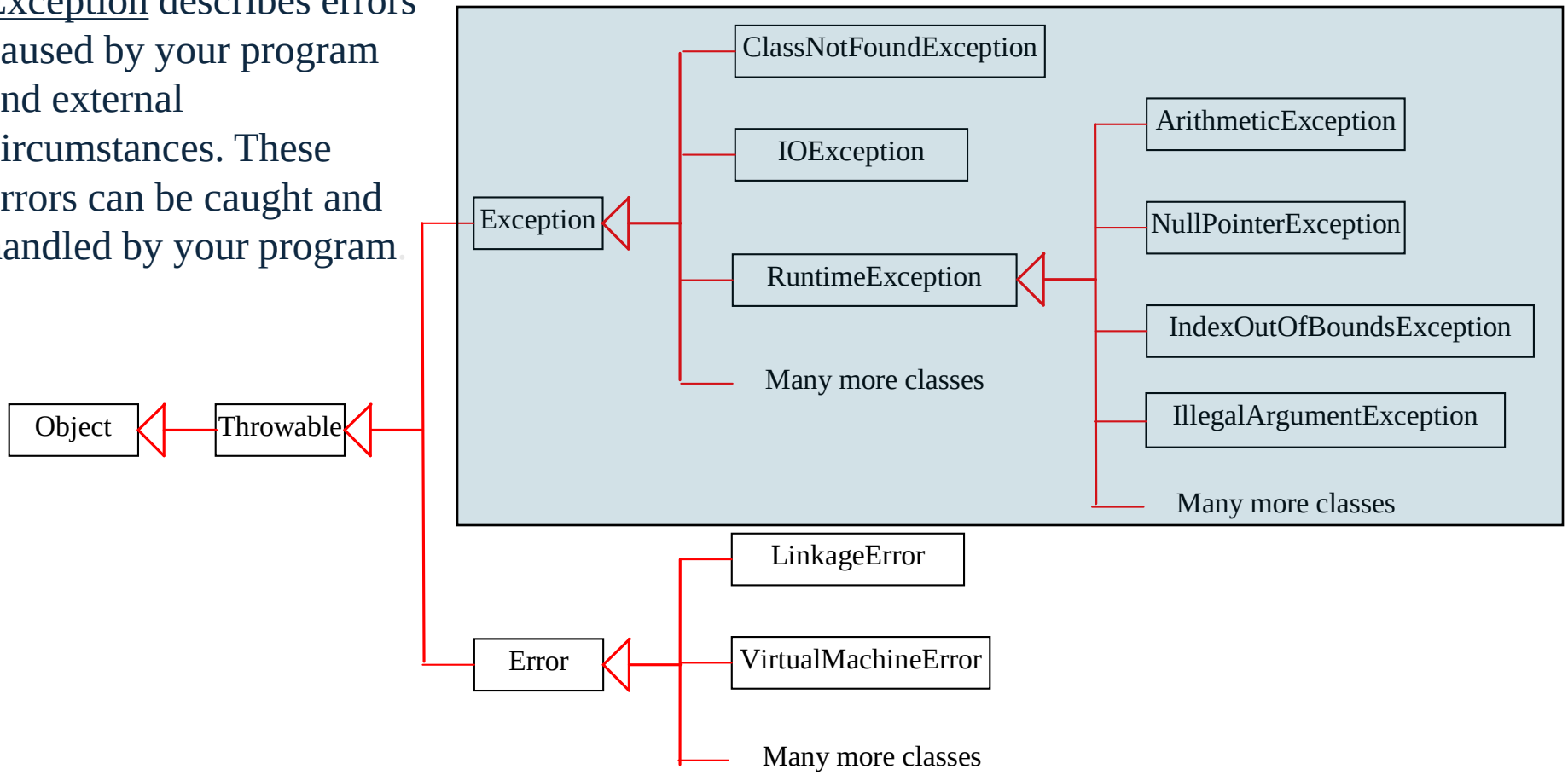
System Errors



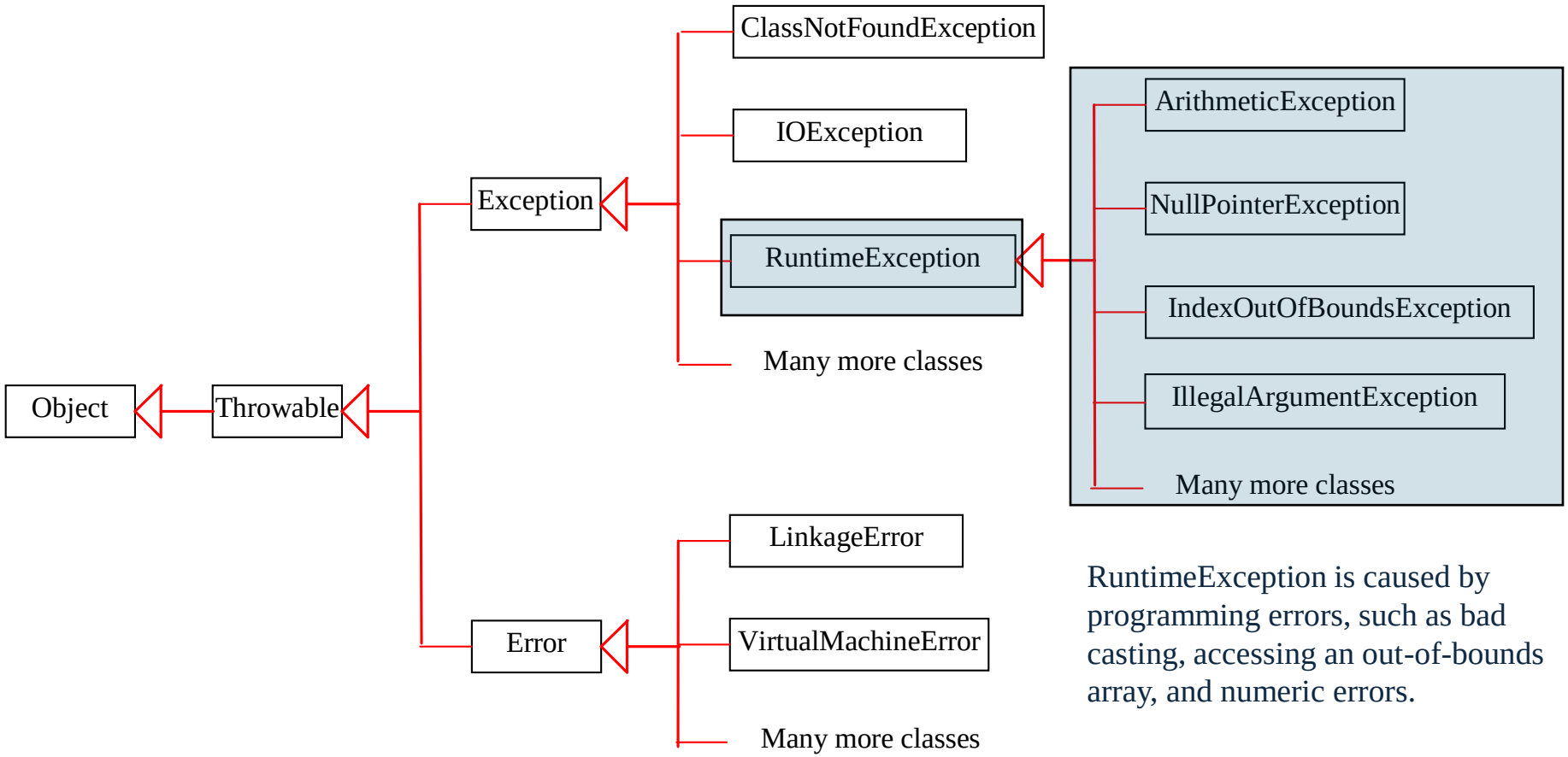
System errors are thrown by JVM and represented in the Error class. The Error class describes internal system errors. Such errors rarely occur. If one does, there is little you can do beyond notifying the user and trying to terminate the program gracefully.

Exceptions

Exception describes errors caused by your program and external circumstances. These errors can be caught and handled by your program.



Runtime Exceptions

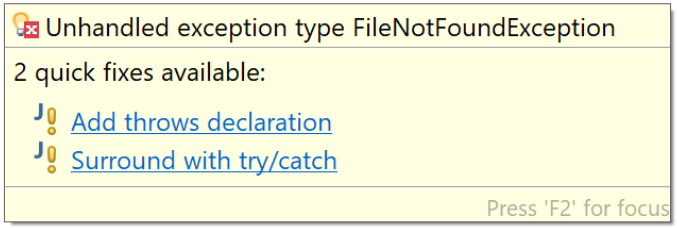


RuntimeException is caused by programming errors, such as bad casting, accessing an out-of-bounds array, and numeric errors.

Exception type

- There are mainly two types of exceptions: checked and unchecked.
- Checked Exception
  - Checked exceptions are checked at compile-time.
  - These exceptions must be either caught or declared to be thrown.
- Unchecked Exception
  - Unchecked exceptions are not checked at compile-time rather they are checked at runtime.
  - These exceptions do not need to be explicitly handled.

```
public static void main(String[] args) {
    FileReader file = new FileReader("data.txt");
```



Checked Exceptions

- Exceptions that are checked at compile time
- If a method throws a checked Exception, then the exception must be handled using a try-catch block and declared in the method signature using the throws keyword.
- Represent invalid conditions in areas outside the immediate control of the program like memory, network, file system, database connection etc.
- Checked exceptions must be either caught by the caller or listed as part of the method signature using the throws keyword.

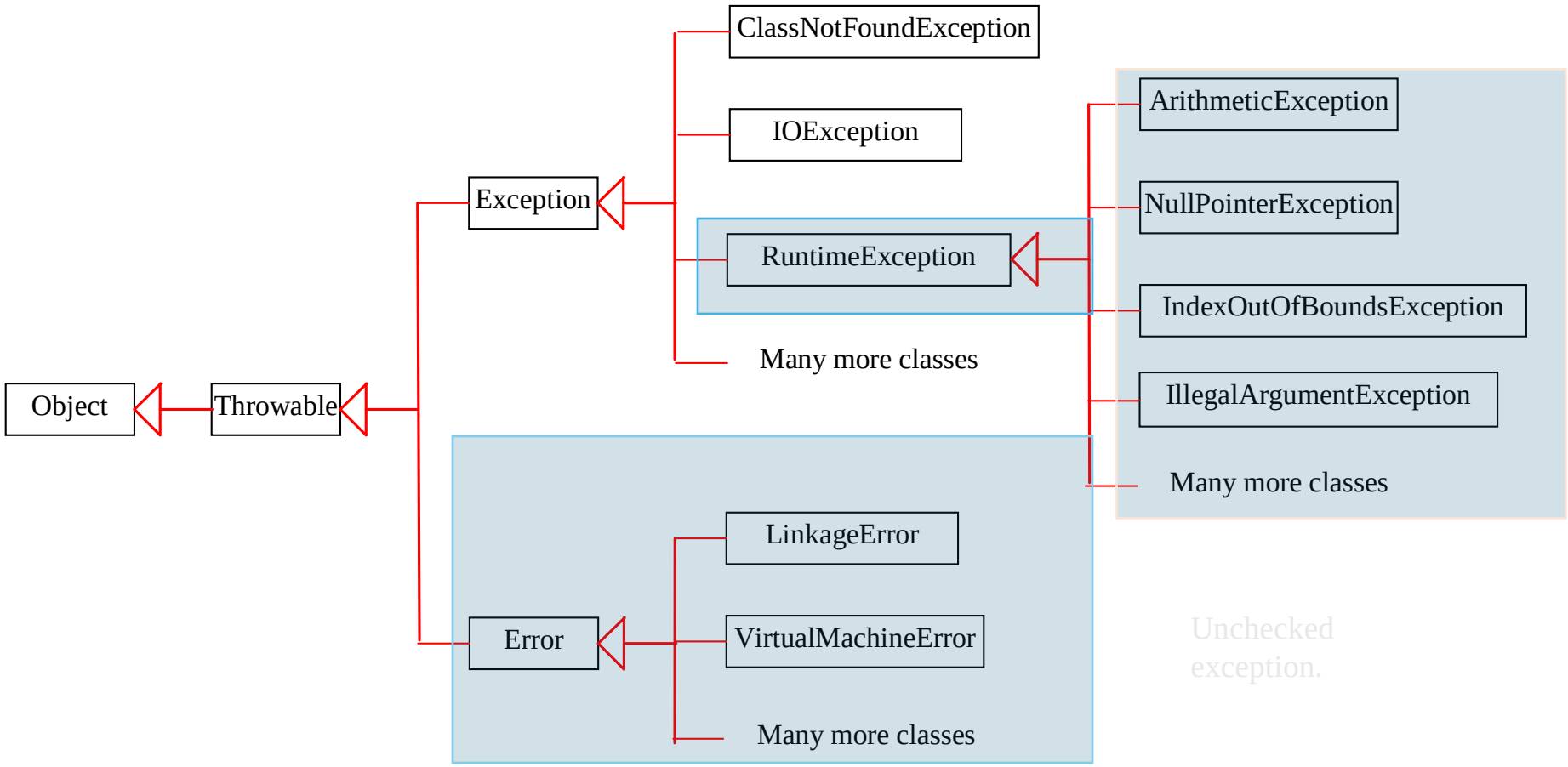
Unchecked Exceptions

- In most cases, unchecked exceptions reflect programming logic errors that are not recoverable.
- Example:
  - A NullPointerException is thrown if you access an object through a reference variable before an object is assigned to it;
  - An IndexOutOfBoundsException is thrown if you access an element in an array outside the bounds of the array.
- These are the logic errors that should be corrected in the program.
- Unchecked exceptions can occur anywhere in the program. To avoid cumbersome overuse of try-catch blocks, Java does not mandate you to write code to catch unchecked exceptions.

Checked Exceptions vs. Unchecked Exceptions

RuntimeException, Error and their subclasses are known as *unchecked exceptions*. All other exceptions are known as *checked exceptions*, meaning that the compiler forces the programmer to check and deal with the exceptions.

Unchecked Exceptions



Java Exception Handling Keywords

- try
  - Java try block is used to enclose the code that might throw an exception. It must be used within the method and must be followed by either catch or finally block.
- catch
  - Java catch block is used to handle the Exception. It must be used after the try block only. Multiple catch block can be used with a single try.
- finally
  - Java finally block is a block that is used to execute important code such as closing connection, stream etc. It follows try or catch block
  - Java finally block is always executed whether exception is handled or not.

Java Exception Handling Keywords

- throw
  - The Java throw keyword is used to explicitly throw an exception.
  - We can throw either checked or unchecked exception in Java by throw keyword.
  - The throw keyword is mainly used to throw custom exception.
- throws
  - The Java throws keyword is used to declare an exception.
  - It allows you to propagate exceptions from a method to its caller, making it the responsibility of the caller to handle the exception.
  - It gives an information to the programmer that there may occur an exception so it is better for the programmer to provide the exception handling code (in the caller) so that normal flow can be maintained.

Example: throw

```
public void checkAge(int age) {

    if (age < 18) {
        throw new IllegalArgumentException("You must be 18 or older");
    }

    System.out.println("You can enter.");
}
```

Example: throws

```
public void readFile() throws IOException {
    FileReader file = new FileReader("data.txt");
}
Void caller()
{try {
    readFile();
} catch (IOException e) {
    System.out.println("File could not be read.");
}}
```

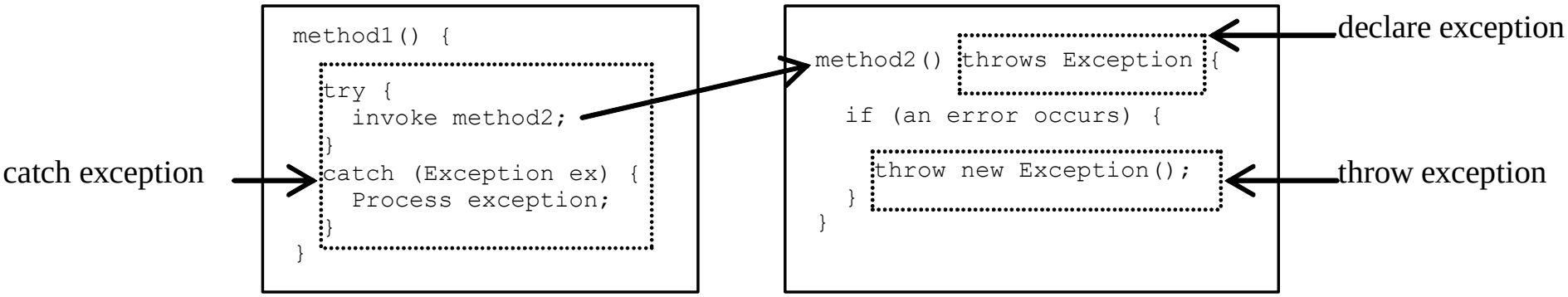
Java Exception Handling Example

```
public class ExceptionHandlingExample {

    //Method that uses 'throws' to declare an exception
    public static void riskyMethod() throws Exception {
        throw new Exception("Exception from riskyMethod");
    }

    public static void main(String[] args) {
        try {
            //Call a method that throws an exception
            riskyMethod();
        } catch (Exception e) {
            //Handle the exception thrown by riskyMethod
            System.out.println("Caught exception: " + e.getMessage());
            throw new RuntimeException("Exception re-thrown from catch block");
//'throw' keyword
        } finally {
            //Code in finally block always executes
            System.out.println("Finally block executed.");
        }
    }
}
```

Declaring, Throwing, and Catching Exceptions



Declaring Exceptions

Every method must state the types of checked exceptions it might throw. This is known as *declaring exceptions*.

public void myMethod() **throws** IOException

or

public void myMethod() **throws** IOException, OtherException

Throwing Exceptions

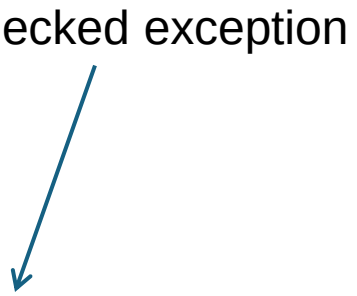
When the program detects an error, the program can create an instance of an appropriate exception type and throw it. This is known as *throwing an exception*. Here is an example,

```
throw new SomeException();
```

or

```
SomeException ex = new SomeException();
throw ex;
```

Throwing Exceptions Example

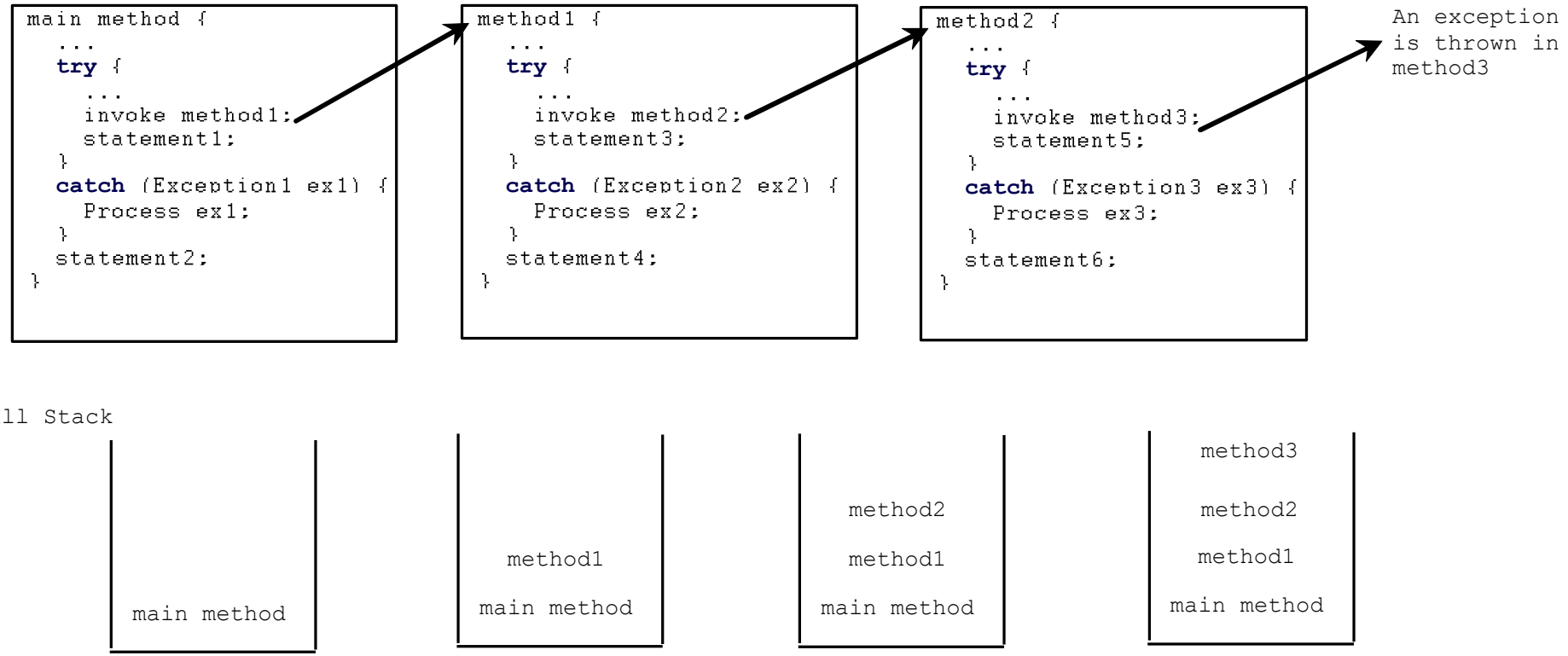



```
 /** Set a new radius */
public void setRadius(double newRadius) throws IllegalArgumentException {
     if (newRadius >= 0)
        radius =  newRadius;
     else
       throw new IllegalArgumentException( "Radius cannot be negative");
}
```

Catching Exceptions

```
try {
  statements;  // Statements that may throw exceptions
}
catch (Exception1 exVar1) {
  handler for exception1;
}
catch (Exception2 exVar2) {
  handler for exception2;
}
...
catch (ExceptionN exVar3) {
  handler for exceptionN;
}
```

Catching Exceptions



Catch or Declare Checked Exceptions

Suppose p2 is defined as follows:

```
void p2() throws IOException {
  if (a file does not exist) {
     throw new IOException("File does not exist");
  }

  ...
}
```

Catching or Declaring Checked Exceptions

- Java forces you to deal with checked exceptions.
- If a method declares a checked exception (i.e., an exception other than Error or RuntimeException), you must invoke it in a try-catch block or declare to throw the exception in the calling method.

Catching or Declaring Checked Exceptions

- For example, suppose that method p1 invokes method p2 and p2 may throw a checked exception (e.g., IOException), you have to write the code as shown in (a) or (b).

```
void p1() {
  try {
    p2();
  }
  catch (IOException ex) {
    ...
  }
}
```

(a)

or

```
void p1() throws IOException {

  p2();

}
```

(b)

What if we have unchecked Exception?

```
void p2() throws NullPointerException {
    throw new NullPointerException("Something went wrong");
}

void caller() {
    p2();  //completely valid as NullPointerException in UnChecked
}
```

Example: Declaring, Throwing, and Catching Exceptions

- ***Objective: This example demonstrates declaring, throwing, and catching exceptions by modifying the setRadius method. setRadius throws an exception if radius is negative.***

```
public class CircleWithException {
  /** The radius of the circle */
  private double radius;

  /** The number of the objects created */
  private static int numberOfObjects = 0;

  /** Construct a circle with radius 1 */
  public CircleWithException() {
    this(1.0);
  }

  /** Construct a circle with a specified radius */
  public CircleWithException(double newRadius) {
    setRadius(newRadius);
    numberOfObjects++;
  }

  /** Return radius */
  public double getRadius() {
    return radius;
  }

  /** Set a new radius */
  public void setRadius(double newRadius)
      throws IllegalArgumentException {
    if (newRadius >= 0)
      radius = newRadius;
    else
      throw new IllegalArgumentException(
        "Radius cannot be negative");
  }

  /** Return numberOfObjects */
  public static int getNumberOfObjects() {
    return numberOfObjects;
  }

  /** Return the area of this circle */
  public double findArea() {
    return radius * radius * 3.14159;
  }
}
```

Rethrowing Exceptions

- Rethrowing an exception is a common practice when a catch block cannot handle a particular exception.
- This allows the exception to be propagated to a higher level where it can be handled appropriately.

```
try {
  statements;
}
catch(SomeException ex) {
  perform operations before exits;
  throw ex;
}
```

Example: Rethrowing Exceptions

```
public static void method2() {
    try {
        method3();
    } catch (ArithmeticException ex) {
        System.out.println("Logging the error...");
        throw ex;   // rethrow
    }
}

public static void main(String[] args) {
    try {
        method2();
    } catch (ArithmeticException ex) {
        System.out.println("Handled in main");
    }
}
```

The `finally` Clause

```
try {
   statements;
}
catch(SomeException ex) {
   handling ex;
}
finally {
   finalStatements;
}
```

animation

Trace a Program Execution

Suppose no exceptions in the statements

```
try {
  statements;
}
catch(SomeException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

animation

Trace a Program Execution

```
try {
  statements;
}
catch(SomeException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The final block is always executed

animation

Trace a Program Execution

```
try {
  statements;
}
catch(SomeException ex) {
  handling ex;
}
finally {
  finalStatements;
}
```

```
Next statement;
```

Next statement in the method is executed

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Suppose an exception of type Exception1 is thrown in statement2

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The exception is handled.

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The final block is always executed.

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

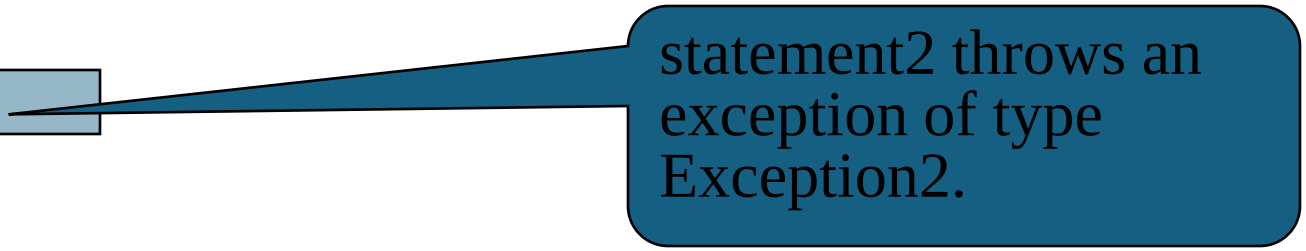
The next statement in the method is now executed.

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

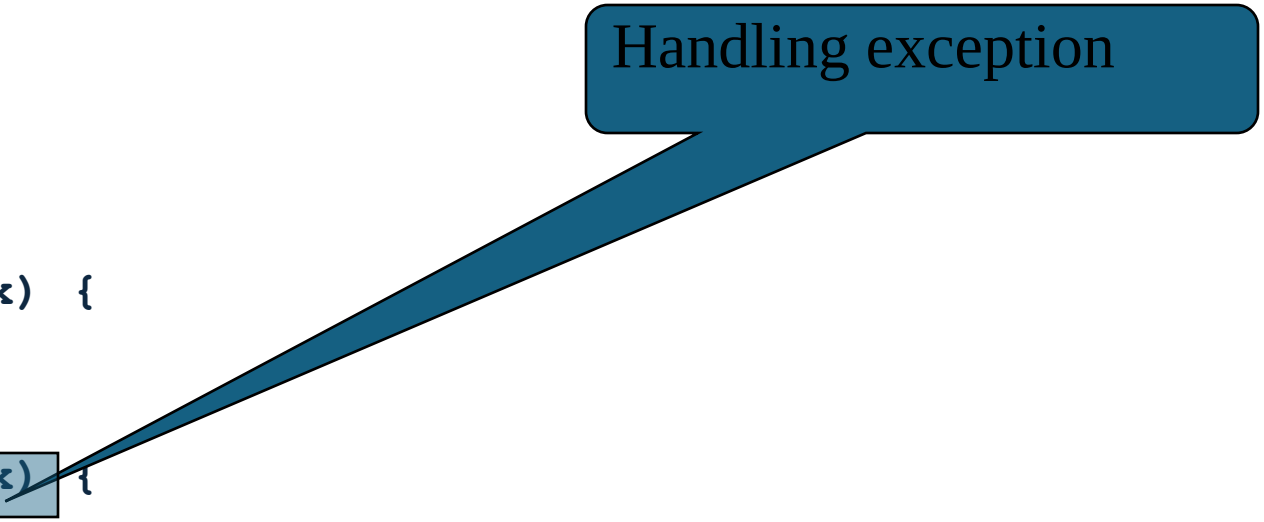


animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

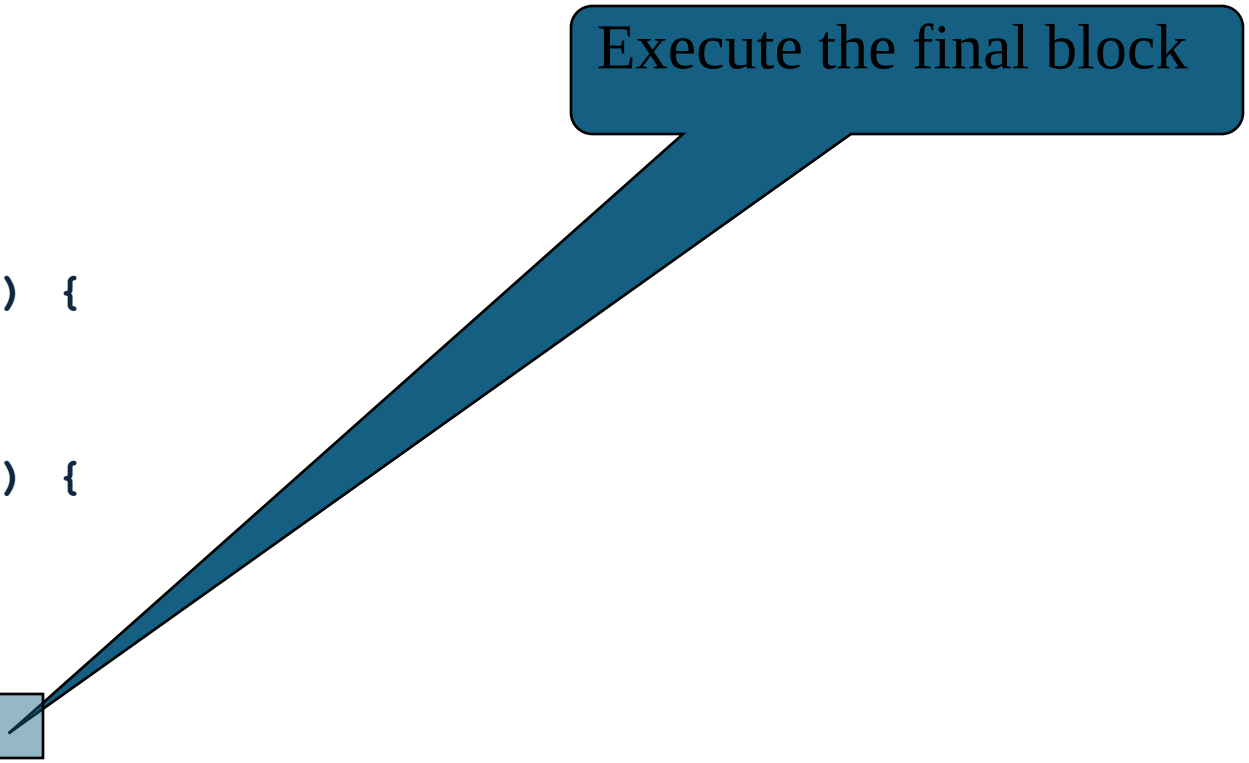


animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```



animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

Rethrow the exception and control is transferred to the caller

Caution When Using Exceptions

- Exception handling separates error-handling code from normal programming tasks, thus making programs easier to read and to modify.
- Be aware, however, that exception handling usually requires more time and resources because it requires instantiating a new exception object, rolling back the call stack, and propagating the errors to the calling methods.

When to Throw Exceptions

- An exception occurs in a method.
- If you want the exception to be processed by its caller, you should create an exception object and throw it.
- If you can handle the exception in the method where it occurs, there is no need to throw it.

When to Use Exceptions

When should you use the try-catch block in the code?

- You should use it to deal with unexpected error conditions.
- Do not use it to deal with simple, expected situations.
- For example, the following code

```
try {
   System.out.println(refVar.toString());
}
catch (NullPointerException ex) {
   System.out.println("refVar is null");
}
```

When to Use Exceptions

is better to be replaced by

```
if (refVar != null)
  System.out.println(refVar.toString());
else
  System.out.println("refVar is null");
```

Defining Custom Exception Classes

- A custom exception in Java is a user-defined exception class that extends either Exception (for checked exceptions) or RuntimeException (for unchecked exceptions).
- Use the exception classes in the API as much as possible.
- Define custom exception classes only if the predefined classes are not sufficient.
- Define a **checked** custom exception by extending Exception (or a subclass of Exception), and an **unchecked** custom exception by extending RuntimeException (or a subclass of RuntimeException).

Custom Exception Class Example

- In the previous `CircleWithException` example, the `setRadius` method throws an exception if the radius is negative.
- If, for example, you want to pass the radius to the handler (caller) instead, you need to create a custom exception class to do that.

Custom Exception Class Example

In the previous CircleWithException example, the setRadius method throws an exception if the radius is negative. Suppose you wish to pass the radius to the handler (caller), you have to create a custom exception class.

```
public class InvalidRadiusException extends Exception {
    private double radius;

    /** Construct an exception */
    public InvalidRadiusException(double radius) {
        super("Invalid radius " + radius);
        this.radius = radius;
     }

    /** Return the radius */
    public double getRadius() {
        return radius;
    }
}
```

Custom Exception Class Example (2)

```
public class Circle {
  /** The radius of the circle */
  private double radius;

  /** The number of the objects created
*/
  private static int numberOfObjects = 0;

  /** Construct a circle with radius 1 */
  public Circle () {
    this(1.0);
  }

  /** Construct a circle with a radius */
  public Circle (double newRadius) {
    try {
      setRadius(newRadius);
      numberOfObjects++;
    }
    catch (InvalidRadiusException ex) {
      ex.printStackTrace();
    }
  }
```

```
  /** Return radius */
  public double getRadius() {
    return radius;
  }

  /** Set a new radius */
  public void setRadius(double newRadius)
      throws InvalidRadiusException {
    if (newRadius >= 0)
      radius =  newRadius;
    else
      throw new
InvalidRadiusException(newRadius);
  }

  /** Return numberOfObjects */
  public static int getNumberOfObjects() {
    return numberOfObjects;
  }

  /** Return the area of this circle */
  public double findArea() {
    return radius * radius * 3.14159;
  }
}
```

```
public class TestCircle {
    public static void main(String[] args) {

        // Valid radius
        Circle c1 = new Circle(5);

        System.out.println("Radius: " + c1.getRadius());
        System.out.println("Area: " + c1.findArea());

        // Invalid radius
        try {
            c1.setRadius(-10);
        } catch (InvalidRadiusException ex) {
            System.out.println("Exception caught!");
            System.out.println(ex.getMessage());
            System.out.println("Invalid radius: " + ex.getRadius());
        }

        System.out.println("Number of objects: " + Circle.getNumberOfObjects());
    }
}
```

Exception Handling Best Practices

- Catch only those exceptions you can handle
  - Avoid catching exceptions that you don’t intend to handle. For example, catching Exception can sometimes lead to hiding bugs.
- Use specific exception types
  - Always try to catch specific exceptions (like FileNotFoundException) instead of generic ones (Exception).
- Do not swallow exceptions
  - If you catch an exception, always handle it properly or log it. Do not just ignore it.
- Use **finally** for resource management
  - Always release resources such as file handles, database connections, and sockets in the finally block.

Assertions

- An assertion is a Java statement that enables you to **assert an assumption about your program.**
- An assertion contains **a Boolean expression** that should be true during program execution.
- Assertions can be used to **assure program correctness and avoid logic errors** during development as a sort of testing.

Declaring Assertions

- An assertion is declared using the Java keyword assert (since JDK 1.4) as follows.

```
assert assertion; or
assert assertion : detailMessage;
```

where `assertion` is a Boolean expression and `detailMessage` is a primitive-type or an Object value.

```
int result = calculate();
assert result >= 0 : "Result should not be negative";
```

Executing Assertions

- When an assertion statement is executed, Java evaluates the assertion.
- If it is false, an **`AssertionError`** will be thrown. This means the program will terminate unless the error is caught and handled.
- The `AssertionError` class has a no-arg constructor and seven overloaded single-argument constructors of type int, long, float, double, boolean, char, and Object.

```
Exception in thread "abc" java.lang.AssertionError: Some Message
```

Executing Assertions

- For the first assert statement with no detail message, the *no-arg constructor* of AssertionError is used.
- For the second assert statement with a detail message, an *appropriate AssertionErrorconstructor* is used to match the data type of the message.
- Since AssertionErroris a subclass of Error, when an assertion becomes false, the program displays a message on the console and terminates.

Executing Assertions: Example

```
public class AssertionDemo {
    public static void main(String[] args) { int i;
     int sum = 0;
       for (i = 0; i < 10; i++) {
       sum += i;
     }
     assert i == 10;
     assert sum > 10 && sum < 5 * 10 : "sum is " + sum;
  }
}
```

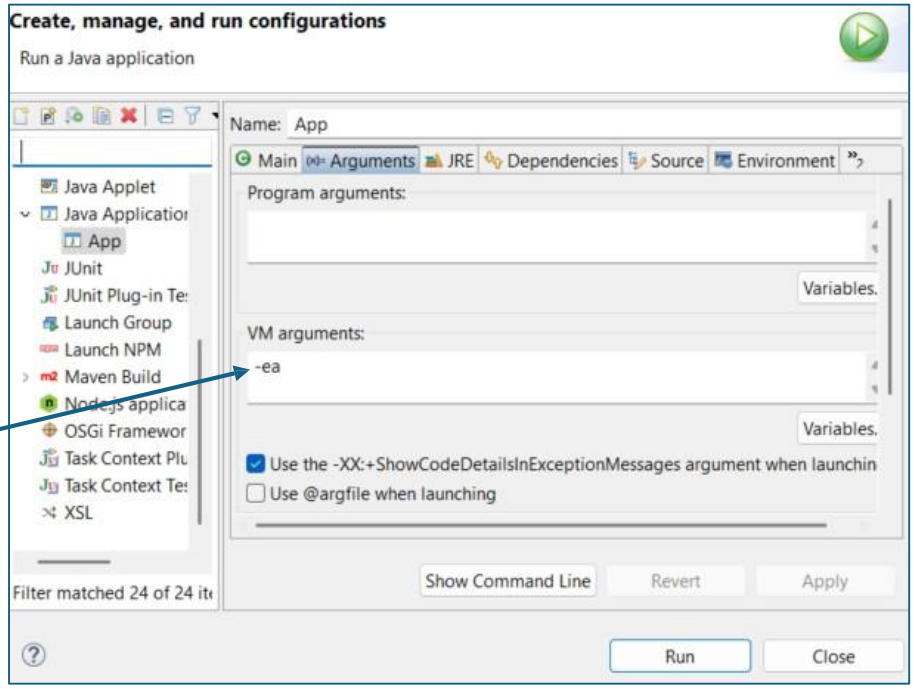
After the loop:
i = 10
sum = 45

Running Programs with Assertions

- By default, the assertions are disabled at runtime.
- To enable it, use the switch –enableassertions, or –ea for short, as follows.

```
java –ea AssertionDemo
```

You need to activate assertions by the JVM by adding –ea in the VM arguments



Running Programs with Assertions

- Assertions can be selectively enabled or disabled at class level or package level. The disabled switch is

  –disableassertions or –da for short.

- For example, the following command enables assertions in package package1 and disables assertions in class Class1.

  java –ea:package1 –da:Class1 AssertionDemo

Exception Handling or Assertions?

- **Assertions** are to assure the correctness of the program logic.
- **Exception handling** deals with unusual circumstances during program execution.
- Assertions are checked at runtime and can be turned on or off at startup time.
- Assertion should not be used to replace exception handling.
- Exception handling addresses **robustness** and assertion addresses **correctness**.
- Like exception handling, assertions are not used for normal tests, but for internal consistency and validity checks.

Exception Handling or Assertions?

- Do not use assertions for argument checking in public methods.
- Valid arguments that may be passed to a public method are considered to be part of the method's contract.
- The contract must always be obeyed whether assertions are enabled or disabled.
- For example, the following code in the Circle class should be rewritten using exception handling.

```
public void setRadius(double newRadius) {
  assert newRadius >= 0;
  radius =  newRadius;
}
```

Exception Handling or Assertions?

- Use assertions to reaffirm assumptions. This gives you more confidence to assure correctness of the program.
- A common use of assertions is to replace assumptions with assertions in the code.

Using Exception Handling or Assertions, cont.

Another good use of assertions is to place assertions in a switch statement without a default case. For example,

```
switch (month) {
  case 1: ... ; break;
  case 2: ... ; break;
  ...
  case 12: ... ; break;
  default: assert false : "Invalid month: " + month
}
```

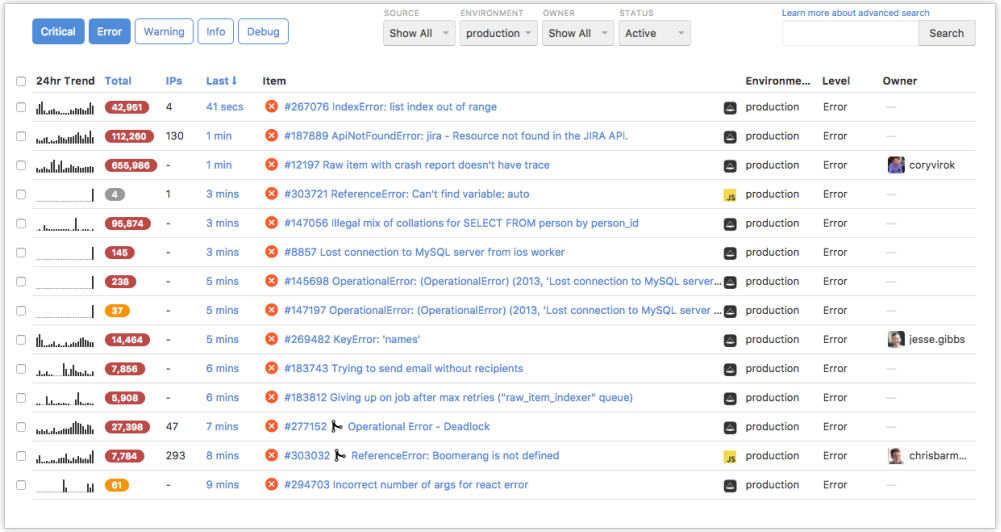
Logging

Intro to logging

•**Definition of Logging**:
The process of recording information about program execution, errors, and other significant events during runtime.

•**Importance**:
It helps developers debug and monitor applications.
Provides insights into program flow and state.
Essential for troubleshooting and improving software reliability.

•**Types of Logging**:
Informational (e.g., system activity)
Debugging (e.g., variable states)
Error (e.g., exceptions)



Key Logging Concepts in Java

Log Levels:

- **SEVERE**: Critical issues (e.g., application crash).
- **WARNING**: Potential problems (e.g., deprecated methods).
- **INFO**: General information (e.g., application startup).
- **DEBUG**: Detailed information for debugging.
- **TRACE**: Very detailed information (used for tracing fine-grained flow).

Loggers:

- A Logger is the library for logging messages.

Logging Example

```
Oct 03, 2025 6:37:41 AM LoggingExample main
INFO: This is an informational message
Oct 03, 2025 6:37:41 AM LoggingExample main
SEVERE: Error occurred: / by zero
```

```
import java.util.logging.*;

public class LoggingExample {
    //Create logger instance
    private static final Logger logger = Logger.getLogger(LoggingExample.class.getName());

    public static void main(String[] args) {
        logger.info("This is an informational message");
        try {
            int result = 10/0;
        } catch (ArithmeticException e) {
            logger.severe("Error occurred: " + e.getMessage());
        }
    }
}
```

Simple Logging Facade for Java (SLF4J)

- Simple Logging Facade for Java (SLF4J) is a logging framework that provides a simple abstraction for various logging frameworks such as `Log4j`, `Logback`, and `java.util.logging`.
- It allows developers to write logging code that is independent of the underlying logging implementation, making it easier to switch between different logging frameworks without changing the application code.

SLF4J Key Features

- **Facade pattern:** SLF4J abstracts away the details of the actual logging framework in use. This allows developers to use a single API (SLF4J) in their code while choosing the logging framework (Logback, Log4j, etc.) at runtime.
- **Flexibility:** With SLF4J, you can plug in any compatible logging framework without modifying the application code.
- **No dependency on logging implementation:** Developers can write logging code once using SLF4J, and the actual logging behavior can be changed by configuring the appropriate logging implementation (like Logback or Log4j).

How it works

- SLF4J acts as an intermediary between the logging API and the actual logging implementation.
- When you write a logging statement using SLF4J, SLF4J delegates that logging call to the configured backend (e.g., Logback or Log4j).

Logback and Log4j

- Logback
  - One of the most widely used logging frameworks in the Java ecosystem.
  - It was developed as the successor to Log4j by the same team behind SLF4J, and it offers better performance, more flexibility, and an easier-to-use configuration system compared to its predecessors.
- Log4j
  - One of the oldest and most widely-used logging frameworks in the Java ecosystem.
  - Developed by the Apache Software Foundation, it became the go-to framework for logging in Java applications.
  - However, due to its limitations in performance and flexibility, Log4j was eventually succeeded by Log4j 2.

Configuration in Maven

```
<dependencies>
   <!-- SLF4J API -->
   <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-api</artifactId>
      <version>VERSION</version>
   </dependency>

<!-- Logback as the Logging Implementation -->
   <dependency>
      <groupId>ch.qos.logback</groupId>
      <artifactId>logback-classic</artifactId>
      <version>VERSION</version>
   </dependency>
</dependencies>
```

Implementation Example

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class SLF4JExample {
    private static final Logger logger =
LoggerFactory.getLogger(SLF4JExample.class);

    public static void main(String[] args) {
        logger.info("Application has started");
        logger.debug("Debugging an issue");
        logger.warn("This is a warning message");
        logger.error("An error has occurred");
    }
}
```

Source Code: logging-01-intro

Logging best practices

1. Log Level Usage:
•Use the appropriate level for each message (e.g., errors at SEVERE, routine information at INFO).

2. Never Log Sensitive Data:
•Be cautious of logging passwords, private keys, or other sensitive user data.

3. Use Structured Logs:
•Consider structured logs (e.g., JSON format) for easier parsing and analysis.

4. Optimize Performance:
•Limit excessive logging in performance-critical sections to avoid overhead.
•Use asynchronous logging where appropriate.

Learning Objective

At the end of this topic, you should be able to understand the following concepts and their roles in software construction

- Dependency management
- Exception handling
- Logging